Exercise 01: Python Introduction

Objective: *Introduction to the Python programming language.*

Read the following pages in *Python for RTL Verification* by Ray Salemi to get familiar with Python.

- Introduction: p. 1-8
- Python Basics: p. 9-138

Or one can use the Open-Source book: "Python For Everybody", chapter 1-6, 8-9 and 14.

Additional reading and examples can be found under:

- `https://github.com/raysalemi/Python4RTLVerification`
- `https://www.youtube.com/playlist?list=PLDAnhhk0KczxDJr5ucQOZ-IcW5yeSa1e4`

0.0.1 Exercise 1 – Functions and Variables: Temperature Converter

Write a function that converts temperatures between Celsius and Fahrenheit.

- Input: value (number) and unit (‘"C"‘ or ‘"F"‘).
- Output: converted value.

Challenge: Extend the function to support Kelvin (‘"K"‘). Use a dictionary of conversion rules instead of ‘if/else‘.

0.0.2 Exercise 2 – Lists and Loops: Even Number Filter

Write a function that takes a list of integers and returns a new list with only the even numbers.

- Use a ‘for‘ loop.

Challenge: Rewrite using a **list comprehension**. Add an optional argument to filter either even or odd numbers.

0.0.3 Exercise 3 – Dictionaries: Word Counter

Write a function that counts how many times each word appears in a given string.

- Use ‘.split()‘ to separate words.
- Store results in a dictionary.

Challenge: Make it case-insensitive and ignore punctuation. Rewrite using **dictionary comprehension** or ‘collections.Counter‘.

0.0.4 Exercise 4 – Control Loops: Multiplication Table

Write a function that prints a multiplication table up to a given number (e.g., 5×5).

- Use nested ‘for‘ loops.

Challenge: Instead of printing, return the table as a **nested list** using list comprehensions.

0.0.5 Exercise 5 – OOP with Python

This exercise will work with OOP and Python by letting you create a base class for shapes and then later derive concrete shapes as circles and squares.

Create a base class: shape Name the base class shape

- Add construcor
- Add empty member function called area

Extend shape to circle

- Extend shape to a derived class called circle which has a radius member variable as an int
- Add a: function void setRadius(int radius); which set the radius
- Add a: function int area(); which computes the area of the circle (Use 3 as a value for Pi)

Extend shape to square

- Extend shape to a derived class called square which has a length member variable as an int
- Add a: function void setLength(int length); which set the length
- Add a: function int area(); which computes the area of the square

Instantiate circle and square

Write a small propgram which instantiates a circle (named c) and a square handle (named s) and set the radius and length and print the computed area

Polymorphism Create a program which creates a list of shapes and instantiates random number of circles and squares. Then make a loop priting each area and finally the total area of all the shapes.

0.0.6 Exercise 6 – Combining Concepts: Student Grades

Create a program that:

- Stores student names and lists of grades in a dictionary.
- Calculates each student's average grade using a loop.
- Finds the student with the highest average.

Challenge: Use a **dictionary comprehension** to compute averages, and ‘max()‘ with ‘key=‘ to find the best student in one line.

0.1 Exercise 7 - Modular Programming

This exercise will go over the fundamentals of modular programming in Python.

- the three classes from subsubsection 0.0.5 into three distinct files and move them to a separate folder named `shapes`.
- Modify the program from subsubsection 0.0.5 such that the classes are imported from the respective source files.

Challange: Create a interface for the `shapes` module by adding exports of all available classes to the `__init__.py` file in the `shapes` folder.